MDPI

Article

Role-Symmetric EAP-EDHOC with Hybrid Post-Quantum Cryptography for 5G Secondary Authentication and Application-Server Access

Adi Panca Saputra Iskandar [1,4], SeongHan Shin [2], Yongho Ko [1], I Wayan Adi Juliawan Pawana [1,4], Hoseok Kwon [1] and Ilsun You [1,3,*]

[1] Department of Cybersecurity, Kookmin University, Seoul 02707, Republic of Korea; adipancaiskandar@gmail.com (A.P.S.I.); koyh0911@kookmin.ac.kr (Y.K.); adijuliawan@kookmin.ac.kr (I.W.A.J.P.)
[2] Cyber Physical Security Research Institute, National Institute of Advanced Industrial Science and Technology (AIST), Tokyo 135-0064, Japan; seonghan.shin@aist.go.jp
[3] KMU Global Research Center for ICT Convergence Security, Kookmin University, Seoul 02707, Republic of Korea
[4] Department of Electrical Engineering, Udayana University, Badung, Bali 80361, Indonesia
* Correspondence: isyou@kookmin.ac.kr

Abstract

5G secondary authentication lets an external data network (DN) authenticate user equipment (UE) through the Extensible Authentication Protocol (EAP), but widely used methods such as EAP-TLS exchange certificate chains over many EAP round-trips and rely on elliptic-curve cryptography that a quantum adversary can break. Moving to post-quantum key encapsulation also breaks the role symmetry of authenticated key exchange, because a key encapsulation mechanism (KEM) implicitly authenticates only the decapsulating party. We propose a role-symmetric hybrid post-quantum EAP-EDHOC (Ephemeral Diffie–Hellman Over COSE) protocol that restores this symmetry: the UE and the DN authentication server each receive exactly one X-Wing (X25519 + ML-KEM-768) encapsulation to a one-time static key, each authenticate with a detached ML-DSA-44 signature, and each rotate and erase their key after every run. The mirrored rotation gives both parties forward secrecy and post-compromise security, and the independent signatures give Key-Compromise Impersonation (KCI) resistance in both directions. The exported Extended Master Session Key bootstraps application-layer keys for login-free server access. We model nine security requirements in ProVerif for seven methods, including a negative control without key erasure. An implementation on Open5GS, UERANSIM, and FreeRADIUS, run through the full 5G core path, shows that the symmetry extends to the traffic: of the 12,259-byte handshake, the UE sends 6,146 bytes and the server 6,113 bytes. The cost, in contrast, is asymmetric and lies in transport fragmentation rather than in cryptography: the proposal computes slightly more than hybrid EAP-TLS but needs half its EAP round-trips.

Keywords: symmetric mutual authentication; asymmetric cryptography; hybrid post-quantum cryptography; EDHOC; 5G secondary authentication; X-Wing KEM; ML-DSA; formal verification

Received:
Revised:
Accepted:
Published:

Copyright: © 2026 by the authors. Submitted to *Symmetry* for possible open access publication under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

Primary authentication through 5G Authentication and Key Agreement (5G-AKA) or EAP-AKA′ proves only that a subscriber is legitimate for the operator, not that it may

use the services of an external data network [1,2], and formal analyses of 5G-AKA have exposed protocol weaknesses [3] and a privacy attack carried over from 4G [4]. 5G therefore defines *secondary authentication* based on the Extensible Authentication Protocol (EAP) [1,5], in which the user equipment (UE) authenticates to a data network (DN) Authentication, Authorization, and Accounting (DN-AAA) server of the service provider; it has drawn dedicated UE–DN frameworks [6] and analyses of the EAP-TLS profile [1,7]. Figure 1 shows the flow: after primary authentication between the Access and Mobility Management Function (AMF) and the Authentication Server Function (AUSF), and slice selection through the Network Slice Selection Function (NSSF) and the Network Repository Function (NRF), the UE runs secondary authentication with the DN-AAA over the Session Management Function (SMF)/User Plane Function (UPF) path, and the DN-AAA bootstraps keys from the Extended Master Session Key (EMSK) toward an application server (AS), which the UE then reaches without a further login.

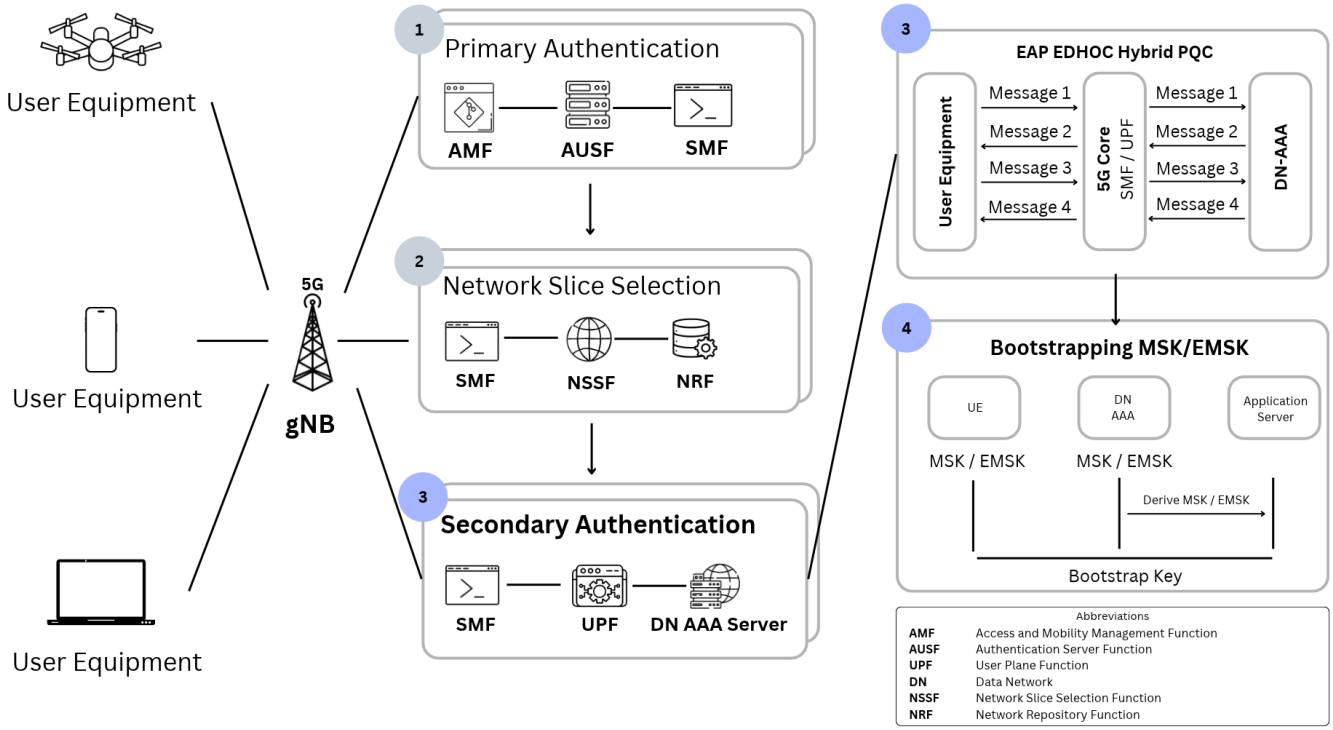


Figure 1. 5G authentication phases and MSK/EMSK key bootstrapping.

EAP-TLS (Transport Layer Security) [7,8] over TLS 1.3 [9] exchanges X.509 certificate chains that add bytes and EAP round-trips on the links of constrained Internet of Things (IoT) and industrial devices. EDHOC (Ephemeral Diffie–Hellman Over COSE) [10,11], a compact key exchange based on the Concise Binary Object Representation (CBOR) and CBOR Object Signing and Encryption (COSE), is carried in EAP by EAP-EDHOC [12] and is markedly more efficient than Datagram TLS (DTLS) 1.3 [13,14], but its elliptic-curve Diffie–Hellman falls to Shor's algorithm [15] and to *harvest-now, decrypt-later* attacks. Hybrid post-quantum cryptography (PQC) is the recommended transition path [16–20], and X-Wing [21,22] is a compact hybrid key encapsulation mechanism (KEM) that combines X25519 with ML-KEM-768. Hybrid-PQC EAP-EDHOC in a real 5G system, and the effect of PQC payloads on Remote Authentication Dial-In User Service (RADIUS) encapsulation [23, 24], remain largely unexplored [25–28].

Symmetry perspective. Replacing Diffie–Hellman with a KEM breaks the role symmetry of key exchange: both Diffie–Hellman parties contribute a share, but a KEM implicitly authenticates only the decapsulator, and EDHOC's methods already mix symmetric (SIG-SIG, STAT-STAT) and mirror-image (SIG-STAT, STAT-SIG) role assignments. Our design restores the symmetry explicitly: each party receives exactly one X-Wing encapsulation to a one-time static key, is authenticated by its own detached ML-DSA-44 signature, and rotates its static key after every run, which replaces the ephemeral key pair as the source of forward secrecy. The protocol's *costs*, by contrast, are strongly asymmetric: cheap in computation and expensive in transport.

The contributions of this work are as follows:

1. A role-symmetric hybrid PQC EAP-EDHOC protocol for 5G secondary authentication, built on X-Wing with a Key-Compromise Impersonation (KCI)-resistant detached ML-DSA-44 signature gate and a symmetric rotation of one-time static keys that gives forward secrecy and post-compromise security without an ephemeral key pair.
2. A ProVerif [29,30] analysis of nine security requirements across the proposal, the four classical EDHOC methods, and two EAP-TLS baselines, with a negative control showing that forward secrecy rests on key erasure.
3. An implementation on Open5GS [31], UERANSIM [32], and FreeRADIUS [33] with open-source code [34], run through the full 5G core path, including EMSK-based key bootstrapping of Object Security for Constrained RESTful Environments (OSCORE) for login-free application-server access.
4. A performance evaluation of computation, communication, EAP/RADIUS fragmentation, and packet loss that quantifies the asymmetry between cryptographic and transport cost.

2. Related Work

Security analysis of classical EDHOC. RFC 9528 has been analysed symbolically by Jacomme et al. [35] (all four methods), Norrman et al. [36], and Kim et al. [37], and computationally by Günther and Mukendi [38], Cremers et al. [39], Cottier and Pointcheval [40], and Ferreira [41]; Pérez et al. [14] survey this work. A recurring finding is that the MAC-based methods (T1–T3) are vulnerable to KCI once the static Diffie–Hellman key leaks [38,39]. These works analyse EDHOC as specified, which remains purely classical.

Post-quantum and hybrid EDHOC. Fraile et al. [25] propose PQ-EDHOC, a fully post-quantum variant using NIST KEMs and signatures, evaluated with microbenchmarks on constrained hardware. A companion work [26] and IETF draft [27] add a signature-free, KEM-based authentication method. Shin et al. [28] embed an ephemeral PQC KEM into EDHOC and omit the optional Message 4, and in follow-up work [42] they strengthen this hybrid EDHOC design toward post-quantum security against KCI. The present paper builds on that line of work and further hardens KCI resistance with a detached ML-DSA-44 gate that is independent of the KEM static keys. None of these proposals target 5G secondary authentication or integrate with EAP/RADIUS, and none combines KCI resistance with post-compromise key rotation.

5G secondary authentication. Gong et al. [6] propose new network functions for UE–DN mutual authentication, and the EAP-TLS profile of TS 33.501 Annex B has been formally analysed [1,7]. These works do not consider PQC or the EAP-over-RADIUS fragmentation that, as we show in Section 6, becomes the bottleneck once PQC payloads are introduced. No prior work combines hybrid PQC, a KCI-resistant gate independent of the KEM static key, formal verification, a live 5G implementation, and EMSK-based key bootstrapping.

Table 1 classifies the seven evaluated methods by the mechanism that authenticates each party. Read row by row, it exposes the role symmetry of each design: Method 0 (SIG-SIG) and Method 3 (STAT-STAT) are role-symmetric, while Methods 1 and 2 are mirror images of each other, each authenticating one party by signature and the other by a static-DH MAC. The static-DH MAC is the mechanism shown to be KCI-vulnerable [38,39]. EAP-TLS is role-symmetric, with entity authentication provided by the `CertificateVerify` signature; its HMAC `Finished` message is a universal key-confirmation step rather than a separate authentication mechanism. Only three methods are post-quantum, and only the proposed method pairs role symmetry with a signature that is both post-quantum and detached from the key-exchange static key. Although the proposal and EAP-TLS Hybrid PQC share the same primitives, they differ in how authentication is framed, which Section 6 shows matters more for latency than the primitives themselves.

Table 1. Authentication mechanisms of the seven evaluated methods.

Method	Initiator authentication	Responder authentication	PQC
EAP-EDHOC Method 0 (SIG-SIG) [10,12]	Signature (Ed25519)	Signature (Ed25519)	No
EAP-EDHOC Method 1 (SIG-STAT) [10,12]	Signature (Ed25519)	Static-DH MAC (HKDF/HMAC-SHA256)	No
EAP-EDHOC Method 2 (STAT-SIG) [10,12]	Static-DH MAC (HKDF/HMAC-SHA256)	Signature (Ed25519)	No
EAP-EDHOC Method 3 (STAT-STAT) [10,12]	Static-DH MAC (HKDF/HMAC-SHA256)	Static-DH MAC (HKDF/HMAC-SHA256)	No
EAP-EDHOC Hybrid PQC (Section 4, over [12])	**Signature (ML-DSA-44, detached SIGMA gate)**	**Signature (ML-DSA-44, detached SIGMA gate)**	**Yes (hybrid)**
EAP-TLS Classic [8,9]	Signature (Ed25519, X.509 `CertificateVerify`) + HMAC `Finished`	Signature (Ed25519, X.509 `CertificateVerify`) + HMAC `Finished`	No
EAP-TLS Hybrid PQC [8,43,44]	Signature (ML-DSA-44, X.509 `CertificateVerify`) + HMAC `Finished`	Signature (ML-DSA-44, X.509 `CertificateVerify`) + HMAC `Finished`	Yes (hybrid)

SIG: signature; STAT: static Diffie–Hellman key; MAC: message authentication code; HKDF: HMAC-based key derivation function.

3. Preliminaries

3.1. 5G Secondary Authentication

EAP [5] defines three roles: the *peer* (the UE), the *authenticator*, which relays messages without understanding the method (the SMF), and the *EAP server*, which runs the method and decides (the DN-AAA) [1,2]. EAP frames travel over the SMF–UPF–DN-AAA path of Figure 1 and are carried in RADIUS between the SMF and the DN-AAA [23,24]. Unlike primary authentication, which the AUSF terminates inside the operator's domain, secondary authentication is terminated by a DN-AAA that the operator does not control and runs during PDU session establishment, before the session is accepted [1,45]. A successful method exports the Master Session Key (MSK) and the Extended MSK (EMSK) [5,46]. The MSK protects the UE–DN session; the EMSK never leaves the peer and the server and serves as a root for cryptographically separated keys through RFC 5295's Usage-Specific Root Keys (USRK). We use it to bootstrap an OSCORE [47] context with an application server without any further login.

3.2. X-Wing Hybrid KEM

X-Wing [21,22] combines X25519 with ML-KEM-768 (FIPS 203) [48,49] and stays secure as long as *at least one* component is unbroken [19,20,50]. Algorithm 1 gives its operations: the public key $ek_{\mathrm{PQ}} \parallel pk_X$ has $1184 + 32$ bytes, and encapsulation merges the ML-KEM and ephemeral X25519 secrets with a SHA3-256 combiner that also binds both public shares, which blocks re-encapsulation and mismatch attacks [39,51]. Our implementation follows the specification [22] exactly and reproduces all of its test vectors; as permitted

for a key that never leaves its device, each party stores the expanded secret key and takes pk_X from its own public key during decapsulation. X-Wing is IND-CCA secure under weaker assumptions than generic combiners [21]. Unlike the negotiated combiners of hybrid TLS 1.3 [43,52], IKEv2 [53], or composite ML-KEM [20,54], it is a fixed instantiation, like PQXDH [55,56] and post-quantum Noise [51], and compact enough for COSE/CBOR. Like any KEM, it implicitly authenticates only the decapsulator; Section 4.5 shows how the protocol compensates.

Algorithm 1 X-Wing hybrid KEM: KeyGen(), Encaps(pk), and Decaps(ct, sk)

KeyGen():
  $seed \leftarrow \mathsf{random}(32)$; $(d, z, sk_X) \leftarrow \mathsf{SHAKE256}(seed, 96)$
  $(dk_{\mathrm{PQ}}, ek_{\mathrm{PQ}}) \leftarrow$ ML-KEM-768.KeyGen_internal(d, z); $pk_X \leftarrow \mathsf{X25519}(sk_X, \mathsf{base})$
  return $pk \leftarrow ek_{\mathrm{PQ}} \,\|\, pk_X$ $(1184 + 32 = 1216$ bytes$)$, $sk \leftarrow dk_{\mathrm{PQ}} \,\|\, sk_X$
Encaps($pk = ek_{\mathrm{PQ}} \,\|\, pk_X$):
  $(ss_{\mathrm{PQ}}, ct_{\mathrm{PQ}}) \leftarrow$ ML-KEM-768.Encaps(ek_{PQ}); $(sk_e, pk_e) \leftarrow \mathsf{X25519.KeyGen}()$
  $ss_X \leftarrow \mathsf{X25519}(sk_e, pk_X)$
  $ss \leftarrow \mathsf{SHA3\text{-}256}(ss_{\mathrm{PQ}} \,\|\, ss_X \,\|\, pk_e \,\|\, pk_X \,\|\, \mathtt{label}_{\text{X-Wing}})$ ▷ combiner
  return $(ss,\ ct \leftarrow ct_{\mathrm{PQ}} \,\|\, pk_e)$
Decaps($ct = ct_{\mathrm{PQ}} \,\|\, pk_e$, $sk = dk_{\mathrm{PQ}} \,\|\, sk_X$):
  $ss_{\mathrm{PQ}} \leftarrow$ ML-KEM-768.Decaps$(ct_{\mathrm{PQ}}, dk_{\mathrm{PQ}})$; $ss_X \leftarrow \mathsf{X25519}(sk_X, pk_e)$
  return $ss \leftarrow \mathsf{SHA3\text{-}256}(ss_{\mathrm{PQ}} \,\|\, ss_X \,\|\, pk_e \,\|\, pk_X \,\|\, \mathtt{label}_{\text{X-Wing}})$

4. Proposed EAP-EDHOC Hybrid PQC Protocol

4.1. Assumptions and Design Rationale

In a provisioning phase (Algorithm 2), the UE (Initiator) and the DN-AAA (Responder) each generate an X-Wing key pair and an ML-DSA-44 signing key and hold the peer's X-Wing public key; the verification keys VK_R and VK_I travel in `Plaintext2/Plaintext3`, bound to the transcript by the signatures. Messages are carried over EAP [5] and RADIUS [23,24] between the pass-through authenticator (SMF/UPF) and the DN-AAA [1], and a Dolev–Yao adversary [57] controls the network. The X-Wing static keys are *one-time* keys: each party announces its next public key early, the UE in EAD_1 and the DN-AAA in EAD_2, bound to the transcript and covered by the signatures, and switches to it, erasing the old secret key, once the run completes. The DN-AAA keeps one record per UE, $(sk_{\mathsf{B}}, pk_{\mathsf{B}}, pk_{\mathsf{A}}, VK_I)$, selected by $KeyID_{\mathsf{B}} = H(pk_{\mathsf{B}})$, which `Message1` carries in the clear, so the outer EAP identity can be an anonymous NAI [58]; each party keeps its keys in a two-slot file written with one `fdatasync`. Authentication follows the sign-and-MAC principle of SIGMA [59], although not the SIGMA-I flow, since ID_CRED_I travels in `Message1` under the Responder's one-time key: X-Wing provides confidentiality and key agreement, and detached ML-DSA-44 signatures [60], independent of the KEM keys, are the sole source of entity authentication, which closes the KCI gap of a MAC-only Method 4 variant [38,39]. We choose ML-DSA-44 (NIST category 2) over ML-DSA-65 to limit fragmentation (1312-byte key, 2420-byte signature), a deliberate size/security trade-off below the category-3 strength of ML-KEM-768.

Algorithm 2 Long-term key generation (provisioning)

At UE: $(sk_{\mathsf{A}}, pk_{\mathsf{A}}) \leftarrow$ X-WING.KeyGen(); $(sk_{\mathsf{SIG,I}}, VK_I) \leftarrow$ ML-DSA-44.KeyGen()
At DN-AAA: $(sk_{\mathsf{B}}, pk_{\mathsf{B}}) \leftarrow$ X-WING.KeyGen(); $(sk_{\mathsf{SIG,R}}, VK_R) \leftarrow$ ML-DSA-44.KeyGen()
return $(pk_{\mathsf{A}}, sk_{\mathsf{A}}, sk_{\mathsf{SIG,I}}, VK_I)$, $(pk_{\mathsf{B}}, sk_{\mathsf{B}}, sk_{\mathsf{SIG,R}}, VK_R)$

4.2. Handshake

Figure 2 shows the exchange inside the 3GPP secondary-authentication flow, with the SMF as EAP authenticator and the DN-AAA as EAP server reached over RADIUS.

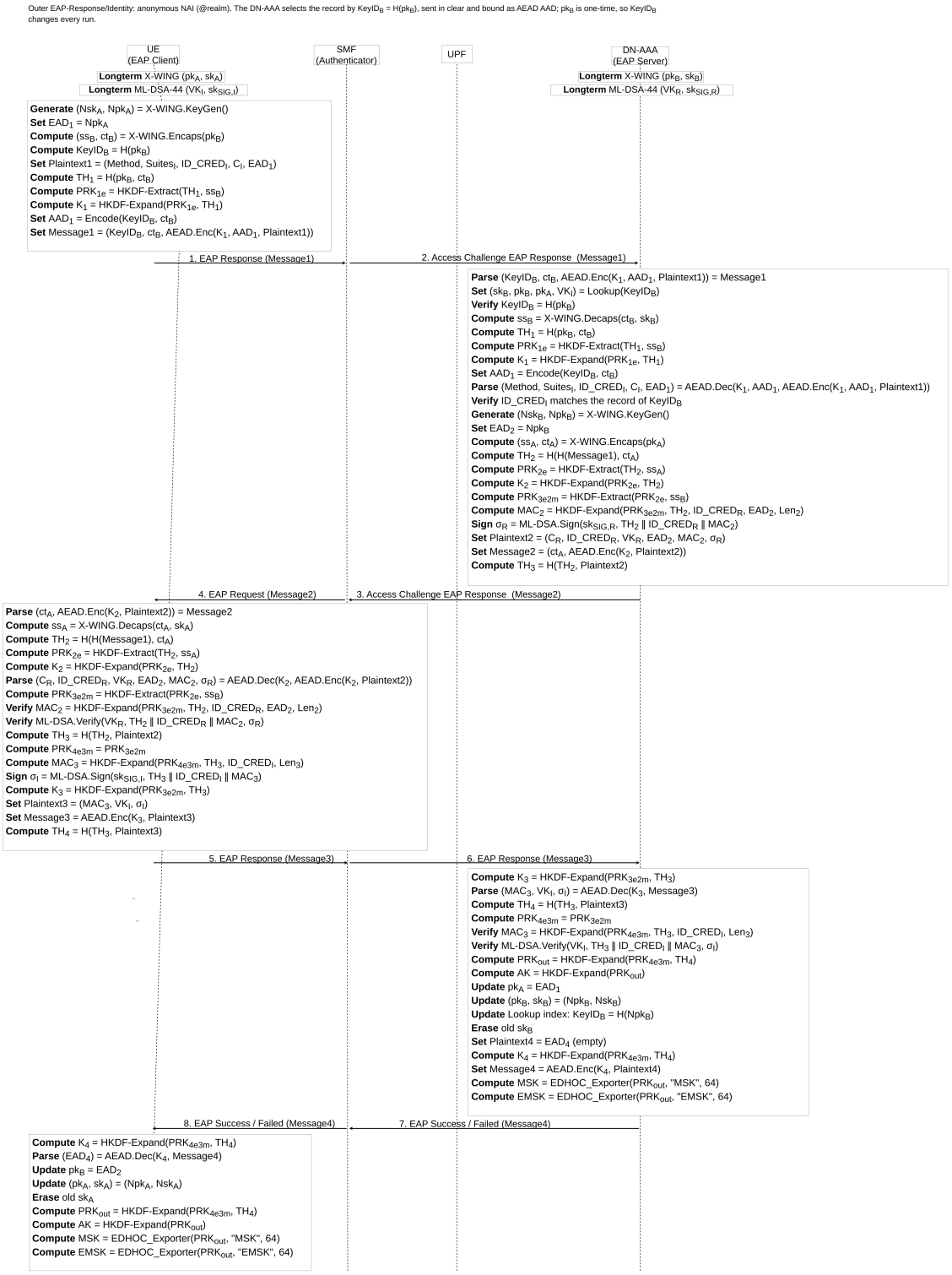


Figure 2. EAP-EDHOC hybrid PQC protocol.

Message 1 (Algorithm 3). The UE generates its next key pair, announces Npk_{A} in EAD_1, encapsulates to the DN-AAA's current one-time key pk_B, and encrypts `Plaintext1`, including ID_CRED_I, under K_1 derived from ss_B. It sends $KeyID_{\mathsf{B}} = H(pk_B)$ in the clear, so that the DN-AAA can select sk_B before decryption, and binds $KeyID_{\mathsf{B}}$ and ct_B to the message as AEAD associated data AAD_1 and through $TH_1 = H(pk_B, ct_B)$. As K_1 depends only on ss_B, ID_CRED_I is exposed only if the current sk_B is compromised before the run.

Algorithm 3 Construction of `Message1` (Initiator)

Require: pk_{B} (current one-time public key of the DN-AAA)
Ensure: Message1
$(Nsk_{\mathsf{A}}, Npk_{\mathsf{A}}) \leftarrow$ X-WING.KeyGen(); $\mathsf{EAD}_1 \leftarrow Npk_{\mathsf{A}}$ ▷ next key of UE
$(ss_{\mathsf{B}}, ct_{\mathsf{B}}) \leftarrow$ X-WING.Encaps(pk_{B}); $KeyID_{\mathsf{B}} \leftarrow H(pk_{\mathsf{B}})$ ▷ key identifier
Plaintext1 $\leftarrow (\mathsf{Method}, \mathsf{Suites}_I, ID_CRED_I, C_I, \mathsf{EAD}_1)$
$TH_1 \leftarrow H(pk_{\mathsf{B}}, ct_{\mathsf{B}})$
$PRK_{1e} \leftarrow$ HKDF-Extract(TH_1, ss_{B})
$K_1 \leftarrow$ HKDF-Expand(PRK_{1e}, TH_1)
$AAD_1 \leftarrow$ Encode$(KeyID_{\mathsf{B}}, ct_{\mathsf{B}})$
Message1 $\leftarrow \big(KeyID_{\mathsf{B}},\ ct_{\mathsf{B}},\ \mathsf{AEAD.Enc}(K_1, AAD_1, \mathsf{Plaintext1})\big)$
return Message1

Message 2 (Algorithm 4). The DN-AAA selects the record by $KeyID_{\mathsf{B}}$, checks $KeyID_{\mathsf{B}} = H(pk_B)$ before decapsulating, opens `Plaintext1` with AAD_1, checks that ID_CRED_I belongs to that record, announces Npk_{B} in EAD_2, and encapsulates to pk_A; ss_A keys `Plaintext2`, so only the holder of sk_A reads the Responder's identity. It signs $TH_2 \,\|\, ID_CRED_R \,\|\, MAC_2$ with a key independent of sk_B, which makes Responder authentication KCI-resistant.

Algorithm 4 EDHOC `Message2` (DN-AAA → UE)

Require: Message1, per-UE records indexed by $KeyID_{\mathsf{B}}$, $sk_{\mathsf{SIG,R}}$, VK_R
Ensure: Message2
$\big(KeyID_{\mathsf{B}},\ ct_{\mathsf{B}},\ \mathsf{AEAD.Enc}(K_1, AAD_1, \mathsf{Plaintext1})\big) \leftarrow$ Message1
$(sk_{\mathsf{B}}, pk_{\mathsf{B}}, pk_{\mathsf{A}}, VK_I) \leftarrow$ Lookup$(KeyID_{\mathsf{B}})$; **verify** $KeyID_{\mathsf{B}} \stackrel{?}{=} H(pk_{\mathsf{B}})$ ▷ before Decaps
$ss_{\mathsf{B}} \leftarrow$ X-WING.Decaps$(ct_{\mathsf{B}}, sk_{\mathsf{B}})$
$TH_1 \leftarrow H(pk_{\mathsf{B}}, ct_{\mathsf{B}})$
$PRK_{1e} \leftarrow$ HKDF-Extract(TH_1, ss_{B})
$K_1 \leftarrow$ HKDF-Expand(PRK_{1e}, TH_1); $AAD_1 \leftarrow$ Encode$(KeyID_{\mathsf{B}}, ct_{\mathsf{B}})$
$(\mathsf{Method}, \mathsf{Suites}_I, ID_CRED_I, C_I, \mathsf{EAD}_1) \leftarrow \mathsf{AEAD.Dec}\big(K_1, AAD_1, \mathsf{AEAD.Enc}(K_1, AAD_1, \mathsf{Plaintext1})\big)$
verify ID_CRED_I belongs to the record of $KeyID_{\mathsf{B}}$
$(Nsk_{\mathsf{B}}, Npk_{\mathsf{B}}) \leftarrow$ X-WING.KeyGen(); $EAD_2 \leftarrow Npk_{\mathsf{B}}$ ▷ next key of DN-AAA
$(ss_{\mathsf{A}}, ct_{\mathsf{A}}) \leftarrow$ X-WING.Encaps(pk_{A})
$TH_2 \leftarrow H\big(H(\mathsf{Message1}),\ ct_{\mathsf{A}}\big)$
$PRK_{2e} \leftarrow$ HKDF-Extract(TH_2, ss_{A})
$K_2 \leftarrow$ HKDF-Expand(PRK_{2e}, TH_2)
$PRK_{3e2m} \leftarrow$ HKDF-Extract$(PRK_{2e}, ss_{\mathsf{B}})$
$MAC_2 \leftarrow$ HKDF-Expand$\big(PRK_{3e2m}, TH_2, ID_CRED_R, EAD_2, Len_2\big)$
$\sigma_R \leftarrow$ ML-DSA.Sign$\big(sk_{\mathsf{SIG,R}},\ TH_2 \,\|\, ID_CRED_R \,\|\, MAC_2\big)$ ▷ **independent of** sk_{B}
Plaintext2 $\leftarrow (C_R, ID_CRED_R, VK_R, EAD_2, MAC_2, \sigma_R)$
Message2 $\leftarrow \big(ct_{\mathsf{A}},\ \mathsf{AEAD.Enc}(K_2, \mathsf{Plaintext2})\big)$
$TH_3 \leftarrow H(TH_2, \mathsf{Plaintext2})$
return Message2

https://doi.org/10.3390/sym1010000

Algorithm 5 EDHOC `Message3` (UE → DN-AAA)

Require: Message2, sk_{A}, ss_{B}, $sk_{\mathsf{SIG,I}}$, VK_I
Ensure: Message3
$(ct_{\mathsf{A}},\ \mathsf{AEAD.Enc}(K_2, \mathsf{Plaintext2})) \leftarrow$ Message2
$ss_{\mathsf{A}} \leftarrow \mathsf{X\text{-}WING.Decaps}(ct_{\mathsf{A}}, sk_{\mathsf{A}})$
$TH_2 \leftarrow H\big(H(\mathsf{Message1}),\ ct_{\mathsf{A}}\big)$
$PRK_{2e} \leftarrow \mathsf{HKDF\text{-}Extract}(TH_2, ss_{\mathsf{A}})$
$K_2 \leftarrow \mathsf{HKDF\text{-}Expand}(PRK_{2e}, TH_2)$
$(C_R, ID_CRED_R, VK_R, EAD_2, MAC_2, \sigma_R) \leftarrow \mathsf{AEAD.Dec}\big(K_2, \mathsf{AEAD.Enc}(K_2, \mathsf{Plaintext2})\big)$
$PRK_{3e2m} \leftarrow \mathsf{HKDF\text{-}Extract}(PRK_{2e}, ss_{\mathsf{B}})$
Verify $MAC_2 \stackrel{?}{=} \mathsf{HKDF\text{-}Expand}(PRK_{3e2m}, TH_2, ID_CRED_R, EAD_2, Len_2)$ ▷ key confirmation
Verify $\mathsf{ML\text{-}DSA.Verify}\big(VK_R,\ TH_2 \,\|\, ID_CRED_R \,\|\, MAC_2,\ \sigma_R\big) \stackrel{?}{=}$ `valid` ▷ **auth gate**
if either check fails: abort
$TH_3 \leftarrow H(TH_2, \mathsf{Plaintext2})$
$PRK_{4e3m} \leftarrow PRK_{3e2m}$ ▷ ss_{A} already entered at PRK_{2e}
store NEXT $\leftarrow (Nsk_{\mathsf{A}}, Npk_{\mathsf{A}}, Npk_{\mathsf{B}} = EAD_2)$ ▷ write-ahead, one `fdatasync`
$MAC_3 \leftarrow \mathsf{HKDF\text{-}Expand}(PRK_{4e3m}, TH_3, ID_CRED_I, Len_3)$
$\sigma_I \leftarrow \mathsf{ML\text{-}DSA.Sign}\big(sk_{\mathsf{SIG,I}},\ TH_3 \,\|\, ID_CRED_I \,\|\, MAC_3\big)$ ▷ **independent of** sk_{A}
$K_3 \leftarrow \mathsf{HKDF\text{-}Expand}(PRK_{3e2m}, TH_3)$
$\mathsf{Plaintext3} \leftarrow (MAC_3, VK_I, \sigma_I)$
$\mathsf{Message3} \leftarrow \mathsf{AEAD.Enc}(K_3, \mathsf{Plaintext3})$
$TH_4 \leftarrow H(TH_3, \mathsf{Plaintext3})$
return Message3

Message 3 (Algorithm 5). The UE verifies MAC_2 and the signature gate σ_R, writes its pending keys (Section 4.4), and authenticates itself in the mirror-image way; `Message3` carries no EAD.

Message 4 and key export (Algorithms 6 and 7). The DN-AAA verifies σ_I, commits its new record, erases the old sk_B, and returns `Message4` with an empty EAD_4 as the protected success indication; the UE switches keys only after verifying it. Both export identical MSK, returned to the 5G core, and EMSK, kept for bootstrapping.

Algorithm 6 EDHOC `Message4` (DN-AAA → UE)

Require: Message3, PRK_{3e2m}, TH_3
Ensure: Message4
$K_3 \leftarrow \mathsf{HKDF\text{-}Expand}(PRK_{3e2m}, TH_3)$
$(MAC_3, VK_I, \sigma_I) \leftarrow \mathsf{AEAD.Dec}(K_3, \mathsf{Message3})$
$TH_4 \leftarrow H(TH_3, \mathsf{Plaintext3})$
$PRK_{4e3m} \leftarrow PRK_{3e2m}$
Verify $MAC_3 \stackrel{?}{=} \mathsf{HKDF\text{-}Expand}(PRK_{4e3m}, TH_3, ID_CRED_I, Len_3)$ ▷ key confirmation
Verify $\mathsf{ML\text{-}DSA.Verify}\big(VK_I,\ TH_3 \,\|\, ID_CRED_I \,\|\, MAC_3,\ \sigma_I\big) \stackrel{?}{=}$ `valid` ▷ **auth gate**
if either check fails: abort
$PRK_{\mathsf{out}} \leftarrow \mathsf{HKDF\text{-}Expand}(PRK_{4e3m}, TH_4);\ AK \leftarrow \mathsf{HKDF\text{-}Expand}(PRK_{\mathsf{out}})$
$pk_{\mathsf{A}} \leftarrow Npk_{\mathsf{A}}$ (from EAD_1) ▷ update next key of UE
$(pk_{\mathsf{B}}, sk_{\mathsf{B}}) \leftarrow (Npk_{\mathsf{B}}, Nsk_{\mathsf{B}})$; **store** the record (one `fdatasync`), index it under $H(Npk_{\mathsf{B}})$, then **erase** the old sk_{B} ▷ write-ahead
$\mathsf{Plaintext4} \leftarrow EAD_4 = \varnothing$ ▷ protected success
$K_4 \leftarrow \mathsf{HKDF\text{-}Expand}(PRK_{4e3m}, TH_4)$
$\mathsf{Message4} \leftarrow \mathsf{AEAD.Enc}(K_4, \mathsf{Plaintext4})$
MSK $\leftarrow$ `EDHOC_Exporter`$(PRK_{\mathsf{out}}, \text{“MSK”}, 64)$; EMSK $\leftarrow$ `EDHOC_Exporter`$(PRK_{\mathsf{out}}, \text{“EMSK”}, 64)$
return Message4

Algorithm 7 `Message4` processing and session-key export (UE)

Require: Message4, PRK_{4e3m}, TH_4
Ensure: MSK, EMSK
$K_4 \leftarrow \mathsf{HKDF\text{-}Expand}(PRK_{4e3m}, TH_4)$; **verify** $\mathsf{AEAD.Dec}(K_4, \mathsf{Message4})$
$pk_{\mathsf{B}} \leftarrow Npk_{\mathsf{B}}$ (from EAD_2) ▷ update next key of DN-AAA
$(pk_{\mathsf{A}}, sk_{\mathsf{A}}) \leftarrow (Npk_{\mathsf{A}}, Nsk_{\mathsf{A}})$; **erase** the old sk_{A} ▷ forward secrecy
$PRK_{\mathsf{out}} \leftarrow \mathsf{HKDF\text{-}Expand}(PRK_{4e3m}, TH_4);\ AK \leftarrow \mathsf{HKDF\text{-}Expand}(PRK_{\mathsf{out}})$
MSK $\leftarrow$ `EDHOC_Exporter`$(PRK_{\mathsf{out}}, \text{“MSK”}, 64)$; EMSK $\leftarrow$ `EDHOC_Exporter`$(PRK_{\mathsf{out}}, \text{“EMSK”}, 64)$
return (MSK, EMSK)

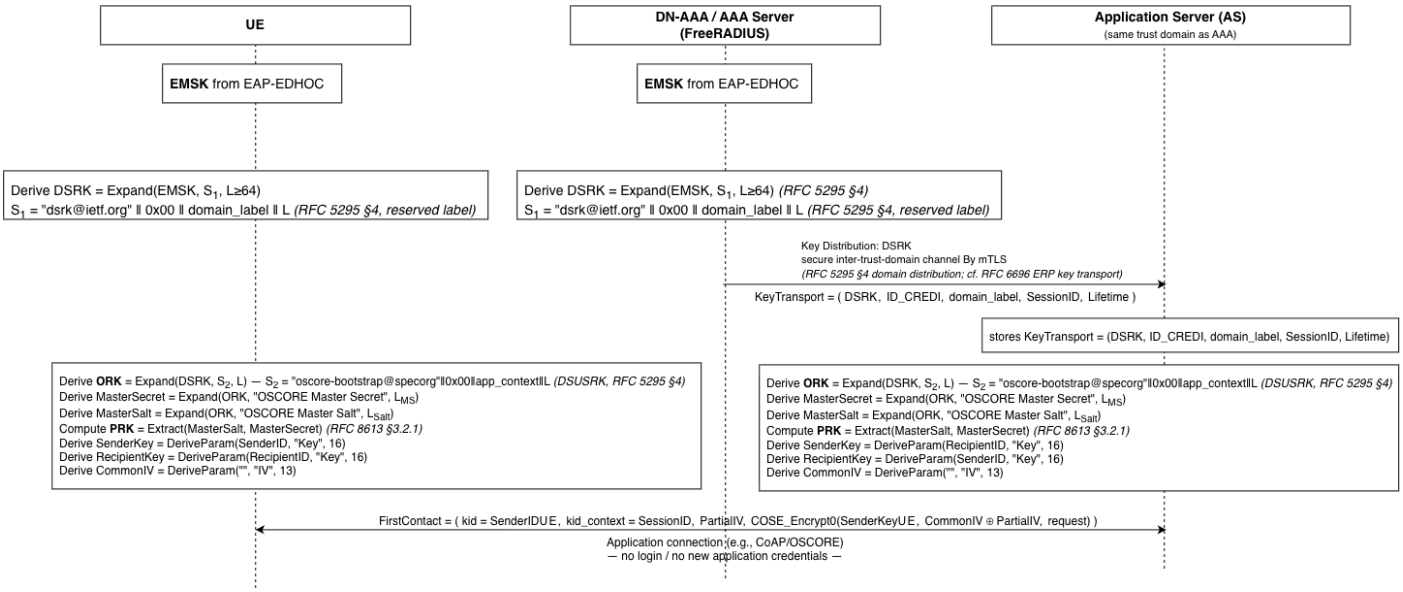


Figure 3. Application-server access via EMSK bootstrapping.

4.3. EMSK-Based Key Bootstrapping

Once secondary authentication succeeds, the UE and the DN-AAA share an identical EMSK. Figure 3 shows how it bootstraps keys that let the UE access the AS directly, and Figure 4 gives the key schedule, which combines RFC 5295's root-key hierarchy [46] with the OSCORE security context of RFC 8613 [47]. Every arrow is a single Expand or Extract call with a unique label, so compromise of one derived key does not disclose keys at another level (cryptographic separation, RFC 5295 Section 3). This phase uses only symmetric-key primitives, so its cost is negligible next to the asymmetric handshake that precedes it (Section 6.5).

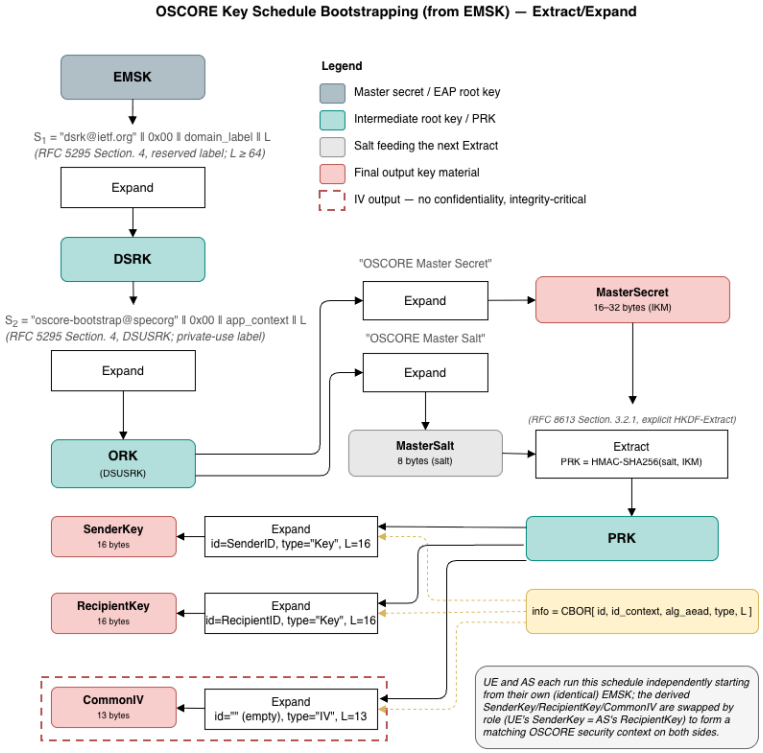


Figure 4. EMSK bootstrapping key schedule.

4.4. Key Synchronization and Recovery

Rotation changes state at both ends, so the order of the durable writes is fixed. The UE writes NEXT $= (Nsk_A, Npk_A, Npk_B)$ before sending `Message3`; the DN-AAA commits its new record after verifying `Message3` and before sending `Message4`, then zeroes the old sk_B, so it keeps no grace key; and the UE promotes NEXT only after verifying `Message4`, the protected success indication that EAP-EDHOC mandates [12]. A UE holding NEXT starts its next run with it. If `Message4` or EAP-Success was lost, the DN-AAA has committed and the run succeeds; if `Message3` was lost, the DN-AAA cannot open `Plaintext1` and answers with an EAP-Failure, and the UE retries once with its old keys. A timeout never triggers this fallback, and the UE keeps NEXT until a run completes, so a forged, unauthenticated EAP-Failure can only delay it, and no case requires re-provisioning. Forward secrecy holds on every path: the session key needs both ss_A and ss_B, and the DN-AAA erases its old sk_B when it accepts a run, so a later compromise cannot recover an accepted session even while the UE still holds its old sk_A, as a dedicated ProVerif model confirms (Section 5); such a retained key exposes at most that run's `Message2` content.

4.5. Symmetry Analysis of the Design

We call a protocol *role-symmetric* if exchanging the Initiator and the Responder maps each party's cryptographic obligations onto those of the other. Table 2 shows that the proposal satisfies this in four respects: (i) two encapsulations, ct_B in `Message1` and ct_A in `Message2`, each feed the key schedule and key the message that delivers them, reproducing with a KEM the two-sided contribution of static Diffie–Hellman; (ii) each party signs its own transcript hash with a detached ML-DSA-44 key that the peer verifies as a mandatory gate, so, unlike the mirror-image methods T1/T2, KCI resistance holds in both directions; (iii) both parties rotate and erase their static keys, announced in EAD_1/EAD_2, which yields forward secrecy (R5) and post-compromise security (R6) without an ephemeral key pair; and (iv) MAC_2 and MAC_3 give mutual key confirmation and identical MSK/EMSK. Three asymmetries are intentional. ID_CRED_I is protected only under the Responder's current one-time key (R2). Only `Message1` carries a key identifier, $KeyID_B$, because only the DN-AAA serves many peers; since pk_B is one-time, $KeyID_B$ changes after every completed run, and the outer EAP identity is an anonymous NAI [58], as recommended for EAP-EDHOC [12]. And `Message2` and `Message3` carry the signatures, although the bytes per direction balance (6,146 B from the UE, 6,113 B from the DN-AAA).

Table 2. Role symmetry of the proposed protocol.

Obligation	Initiator (UE)	Responder (DN-AAA)
Static encapsulation received	ct_A to pk_A (in `Message2`)	ct_B to pk_B (in `Message1`)
Secret fed to key schedule	$ss_A \rightarrow PRK_{2e}$	$ss_B \rightarrow PRK_{3e2m}$
Key-confirmation MAC sent	MAC_3	MAC_2
Detached signature (gate)	σ_I over TH_3, verified by DN-AAA	σ_R over TH_2, verified by UE
Static-key rotation and erasure	Npk_A in EAD_1, old sk_A erased	Npk_B in EAD_2, old sk_B erased
Session-key output	MSK, EMSK	MSK, EMSK
Intentional asymmetry	ID_CRED_I sent under the current pk_B-derived key	record selected by $KeyID_B$

5. Formal Security Verification Using ProVerif

We analyse the protocol with ProVerif [29,30] in the Dolev–Yao model [57,61], following prior EDHOC analyses [35–37,41,62] and ISO/IEC 29128 [63]. The verification targets nine requirements:

R1 **Mutual authentication**: each party is injectively authenticated to the other (agreement on the transcript).

R2 **Identity concealment**: the persistent identities ID_CRED_I and ID_CRED_R are not learned by the adversary.

R3 **Confidentiality**: application data and session keys remain secret.

R4 **Secure key exchange**: MSK/EMSK are secret and agreed.

R5 **Forward secrecy**: past session keys remain secret when the long-term state held after the session (signing keys and current static keys) leaks.

R6 **Post-compromise security**: after a static key is compromised, security is restored once the rotated Npk_A/Npk_B is in use.

R7 **AEAD protection** (Authenticated Encryption with Associated Data): plaintexts are integrity- and confidentiality-protected.

R8 **Quantum resilience**: security holds against an adversary that can solve discrete logarithms, i.e., when X25519 and Ed25519 are broken (modelled by a quantum oracle).

R9 **KCI resistance**: compromise of a party's static KEM key does not allow impersonation of the peer.

Table 3 maps the requirements to queries. We model seven variants, T0–T3, the proposal, and the two EAP-TLS baselines as certificate-authenticated TLS 1.3 [9] in the EAP-TLS 1.3 profile [8], each in three settings: *classical*, where an active attacker learns all long-term keys in phase 1; *quantum*, which adds an oracle, available in every phase, that returns X25519 discrete logarithms and Ed25519 signing keys, modelling Shor's algorithm against active attacks and recorded traffic, with no such oracle for ML-KEM-768 or ML-DSA-44; and *KCI*, where one party's long-term keys are known from the start. X-Wing is modelled with separate halves, so the quantum setting checks the hybrid property rather than assuming it, and the DN-AAA retrieves its key from a table indexed by $KeyID_B$, with $KeyID_B$ and ct_B as associated data of `Message1`. A property is fully secure (●) if it holds in both settings and partial (◐) if it holds only classically. For the proposal, phase 1 leaks the state a device holds after the run (the rotated X-Wing keys and both signing keys); a key-rotation model checks post-compromise security, a negative control (`edhoc_m4v4_fs_noerase.pv`, one session suffices) also leaks the unerased old keys, and a recovery model (`edhoc_m4v4_recovery.pv`) lets the UE keep its old sk_A, as after a lost `Message4`. All models target ProVerif 2.05 and are publicly available [34].

Table 3. Security requirements and verification queries.

Security Requirement	**Verification Query**
Mutual authentication (R1)	Q1: `event(UE_send_M1(M1))` Q2: `inj-event(UE_recv_M2(...))` ⇒ `inj-event(DN_send_M2(...))` Q3: `inj-event(DN_recv_M3(...))` ⇒ `inj-event(UE_send_M3(...))` Q4: `inj-event(UE_recv_M4(...))` ⇒ `inj-event(DN_send_M4(...))`
Identity concealment (R2)	Q5: `attacker(ID_CREDI)`; Q5b: `attacker(ID_CREDR)`
Confidentiality (R3)	Q6: `attacker(app_data_UE)` Q7: `attacker(app_data_DN)`
Secure key exchange (R4)	Q8: `attacker(MSK_test)` Q9: `attacker(EMSK_test)`
Forward secrecy (R5)	Q8 && Q9 after `phase 1` leaks the post-run state; no-erasure control must fail
Post-compromise security (R6)	Q10: `attacker(MSK_test2)` && Q11: `attacker(EMSK_test2)` after the old static keys are revealed
AEAD protection (R7)	Q2 && Q3 && Q4 (implicit via the `AEAD_dec` rewrite)
Quantum resilience (R8)	Q8 && Q9 in the quantum setting (X25519 discrete-log and Ed25519 key-recovery oracle)
KCI resistance (R9)	Q2 && Q4 with the UE's long-term keys leaked; Q3 with the DN's long-term keys leaked

```
Verification summary:
• Query  not event(UE_send_M1(W0,m)) is false.
• Query  inj-event(UE_recv_M2(W0,x,t,a,b))
      ==> inj-event(DN_send_M2(W0,x,t,a,b)) is true.
• Query  inj-event(DN_recv_M3(W0,x,t,a,b))
      ==> inj-event(UE_send_M3(W0,x,t,a,b)) is true.
• Query  inj-event(UE_recv_M4(W0,x,t,a,b))
      ==> inj-event(DN_send_M4(W0,x,t,a,b)) is true.
• Query  not attacker_p1(ID_CREDI[]) is true.
• Query  not attacker_p1(ID_CREDR[]) is true.
• Query  not attacker_p1(app_data_UE[]) is true.
• Query  not attacker_p1(app_data_DN[]) is true.
• Query  not attacker_p1(MSK_test[]) is true.
• Query  not attacker_p1(EMSK_test[]) is true.
• Query  not attacker(MSK_test2[]) is true.
• Query  not attacker(EMSK_test2[]) is true.
```

Figure 5. ProVerif summary of the proposed protocol.

Figure 5 shows the ProVerif output (Q1–Q9 from `edhoc_m4v4_classical.pv`, Q10–Q11 from `edhoc_m4v4_pcs.pv`). Honest runs complete (Q1), the injective correspondences give mutual authentication in both directions (R1), and after the post-run state leaks the attacker learns neither the identities, the application data, nor MSK/EMSK (R2–R5); a later session under the rotated keys regains secrecy (R6). In the negative control Q5–Q9 fail, so forward secrecy and identity concealment rest on erasure, whereas in the recovery model they hold although the UE keeps its old key. As authentication rests on signatures independent of the KEM keys, leaking sk_A/sk_B breaks neither authentication nor KCI resistance (R9), unlike a static-DH MAC variant of Method 4 [38,39].

Table 4 compares the variants. T0–T3 and EAP-TLS Classic hold R1–R5, R7, and R9 only classically and fail R6 and R8; in their quantum models every query turns false. EAP-TLS Hybrid PQC keeps its guarantees against a quantum adversary, consistent with hybrid combiners [20,21], the EUF-CMA security of ML-DSA-44 [60], and TLS 1.3 analyses [62], but, like every baseline, fails R6 because its specification defines no static-key rotation. Only the proposal is fully secure on all nine requirements, with forward secrecy and identity concealment conditional on key erasure.

Table 4. ProVerif results for all seven methods.

Security Requirement	EAP-EDHOC Method 0 [10,12]	EAP-EDHOC Method 1 [10,12]	EAP-EDHOC Method 2 [10,12]	EAP-EDHOC Method 3 [10,12]	EAP-EDHOC Hybrid (Sec. 4)	EAP-TLS Classic [8]	EAP-TLS Hybrid PQC [8,43]
R1 Mutual authentication	◐	◐	◐	◐	●	◐	●
R2 Identity concealment	◐	◐	◐	◐	●[†]	◐	●
R3 Confidentiality	◐	◐	◐	◐	●	◐	●
R4 Secure key exchange	◐	◐	◐	◐	●	◐	●
R5 Forward secrecy	◐	◐	◐	◐	●	◐	●
R6 Post-compromise security	○	○	○	○	●	○	○
R7 AEAD protection	◐	◐	◐	◐	●	◐	●
R8 Quantum resilience	○	○	○	○	●	○	●
R9 KCI resistance	◐	◐	◐	◐	●	◐	●

○ not secure; ◐ partial (secure only against a classical adversary); ● fully secure (classical *and* quantum adversary). Every entry is a ProVerif result, except the ○ marks on R6 for the six baselines, whose specifications define no static-key rotation. [†] ID_CRED_I is sent in `Message1` under a key derived from the DN-AAA's current one-time key. It stays secret after that key is erased, but is exposed if the current sk_B is compromised before the run. The outer EAP identity is an anonymous NAI; unlinkability of runs through $KeyID_B$ is not part of the verified properties.

6. Performance Evaluation

6.1. Testbed and Cryptographic Cost

Experiments ran in an Ubuntu 24.04 LTS VM (16 GB RAM, 12 vCPUs) on an Intel Core i9-14900K host, with Open5GS 2.7.2 [31], UERANSIM 3.2.7 [32], and FreeRADIUS 3.2.6 [33]

co-located over loopback, so the network figures are a lower bound. Figure 6 shows the components we modified or added; code and scripts are public [34]. The EDHOC methods use libsodium [64], mbedTLS [65], and the PQClean [66] ML-KEM-768 and ML-DSA-44. The EAP-TLS 1.3 baselines [8] use OpenSSL 3.0.13 (Classic) and OpenSSL 3.2.6 with liboqs 0.16 [67] and oqs-provider 0.12 [68] (Hybrid PQC: X25519MLKEM768 [43] with ML-DSA-44 certificates [44,69], checked at run time); OpenSSL implements RFC 8446, which RFC 9846 [9] obsoletes compatibly and which forbids key-share reuse, so every handshake generates fresh shares. We measure on two paths. A RADIUS benchmark client, acting as EAP peer and RADIUS client (RFC 2865/3579 [23,24]) against the same FreeRADIUS responder, compares all seven methods, including EAP-TLS, for which UERANSIM has no peer. The proposal also runs through the full path UE → gNB → AMF → SMF → UPF → FreeRADIUS (Section 6.3), for which we enabled `SCTP_NODELAY` on UERANSIM's NGAP socket, as Open5GS does, since Nagle's algorithm otherwise added about 160 ms to every PDU session after the first. Table 5 counts the operations of each party, with every key generation inside the handshake. The proposal costs 0.985 ms, split evenly (0.492 ms per party) because both run mirror-image operations, against 0.894 ms for EAP-TLS Hybrid PQC, which is why its computation bar in Figure 7 (left) is the highest. Both spend 0.566 ms on two ML-DSA-44 signatures and verifications, and EAP-TLS adds 0.119 ms of certificate checks, but the proposal performs six key generations against three: besides the share inside each X-Wing encapsulation, each party generates its next one-time X-Wing key pair for rotation (0.116 ms), and its encapsulation in each direction adds 0.094 ms of key exchange. In process, with key schedule, AEAD, and serialisation, the two parties spend 1.131 ms. In Figure 7 (right), "Initiator" is the UE's local processing and "Responder" the time it waits for replies, including the RADIUS round-trips and the DN-AAA's key storage.

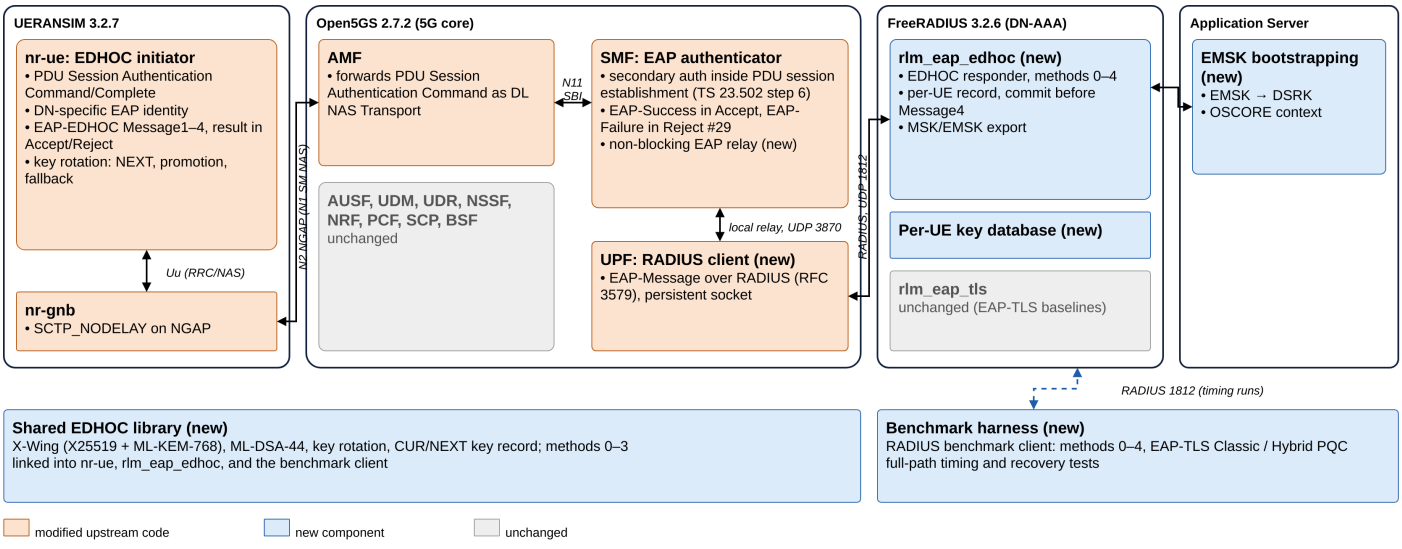


Figure 6. Testbed components: modified, new, and unchanged.

Table 5. Cryptographic operations (Initiator/Responder) and compute time per handshake.

	X25519		ML-KEM-768						Compute (ms)		
Method	**KeyGen**	**DH**	**KeyGen**	**Encaps**	**Decaps**	**Sign**	**Verify**	**KeyGen**[b]	**I**	**R**	**Total**
Classical level (X25519, Ed25519)											
EAP-EDHOC Method 0	1/1	1/1	–	–	–	1/1	1/1	2	0.091	0.091	0.182
EAP-EDHOC Method 1	1/1	2/2	–	–	–	1/0	0/1	2	0.087	0.105	0.192
EAP-EDHOC Method 2	1/1	2/2	–	–	–	0/1	1/0	2	0.105	0.087	0.192
EAP-EDHOC Method 3	1/1	3/3	–	–	–	0/0	0/0	2	0.101	0.101	0.202
EAP-TLS Classic	1/1	1/1	–	–	–	1/1	2/2	2	0.121	0.121	0.241
Hybrid PQC level (X25519 + ML-KEM-768, ML-DSA-44)											
Proposal	2/2[a]	2/2	1/1[a]	1/1	1/1	1/1	1/1	6	0.492	0.492	0.985
EAP-TLS Hybrid PQC	1/1	1/1	1/0	0/1	1/0	1/1	2/2	3	0.467	0.427	0.894

Signatures use Ed25519 (classical) and ML-DSA-44 (hybrid PQC); for EAP-TLS each verification count is `CertificateVerify` + certificate. Compute times use the mean primitive latencies over 3000 iterations (ML-DSA-44 signing, the costliest, 0.223 ms). [a] Per party, one X25519 key pair is the ephemeral share inside X-Wing encapsulation; the other X25519 key pair and the ML-KEM-768 key pair form the next one-time X-Wing key pair (key rotation). In Methods 1–3 a static-DH authentication costs one DH on *each* side. [b] Key generations per handshake, both parties (X25519 and ML-KEM-768).

6.2. End-to-End Latency, Message Size, and Framing

Over the RADIUS path (200 loss-free handshakes per method), the proposal completes in 6.383 ms on average (median 6.284 ms), against 8.273 ms (7.847 ms) for EAP-TLS Hybrid PQC measured alternately in the same session, a median difference of 1.563 ms (bootstrap 95% interval $[1.436, 1.707]$ ms). In an earlier session, EAP-TLS Classic took 3.483 ms (3.101 ms) and Methods 0–3 0.996–1.277 ms (0.954–1.017 ms; 1.021–1.039 ms over pooled runs of 10,000); as absolute times vary across sessions of our VM, we compare methods only within one session. The differences stem from transport, not computation (Figure 8).

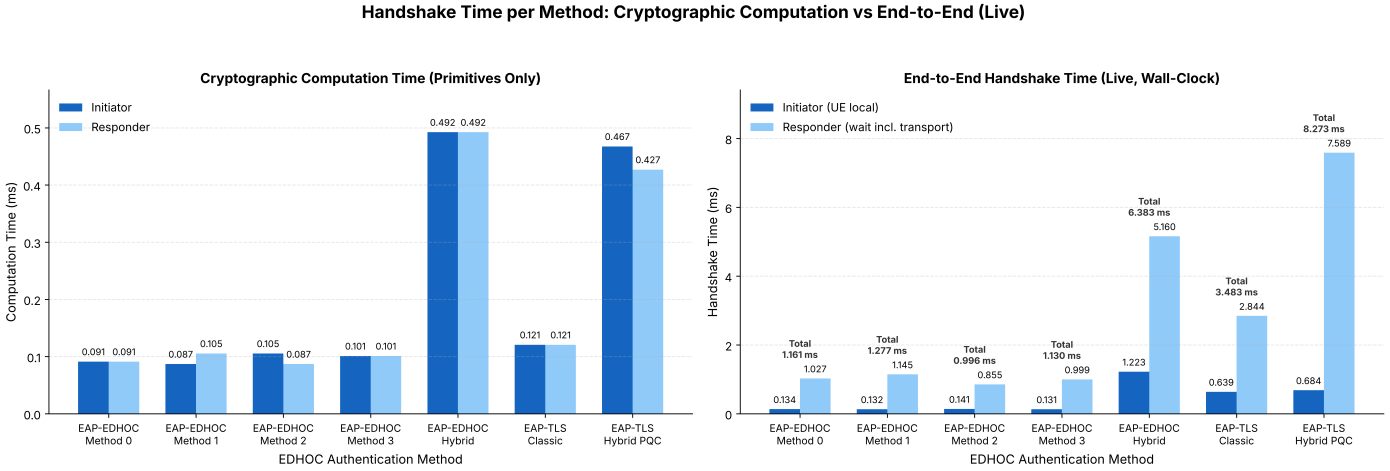


Figure 7. Computation cost (**left**) and end-to-end handshake time (**right**).

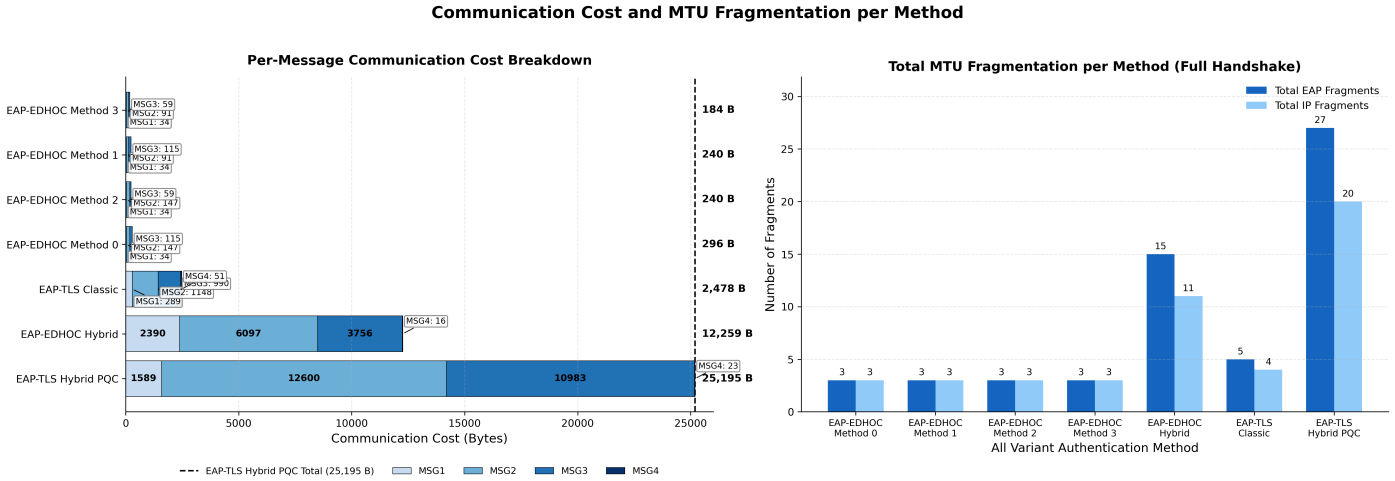


Figure 8. Message size and MTU fragmentation.

Each X-Wing element adds 1.1–1.2 KB and each ML-DSA-44 signature 2.4 KB, so `Message1` (2,390 B, including the 32-byte $KeyID_B$), `Message2` (6,097 B), and `Message3` (3,756 B) split into 3, 7, and 4 EAP fragments of 253-byte RADIUS attributes [24], while `Message4` (16 B) fits in one: 12,259 B in 15 round-trips, balanced between the UE (6,146 B) and the DN-AAA (6,113 B), against 25,195 B in 28 for EAP-TLS Hybrid PQC, which carries ML-DSA-44 inside X.509 chains, and 2,478 B in 6 for EAP-TLS Classic. Each RADIUS round-trip costs about 0.3 ms on loopback for both protocols, so the 13 round-trips the proposal saves outweigh its 0.09 ms of extra computation and its two durable writes, by a margin that grows with the SMF–DN-AAA round-trip time (RTT): a linear projection $T_{\text{loopback}} + n_{\text{RT}} \cdot \text{RTT}$ gives 156 against 288 ms at 10 ms. The two `fdatasync` calls on the critical path, the UE's write of NEXT and the DN-AAA's commit, cost 1.10 ms against a `tmpfs` reference (bootstrap 95% interval $[1.04, 1.15]$ ms; 0.53 ms for the UE and 0.61 ms for the DN-AAA), down from 7.9 ms when each key had its own file. The DN-AAA keeps its $KeyID_B$ index in memory and rebuilds it from the record slots at start-up (0.46 s for 10,000 records with a warm page cache), so the lookup adds no write to the critical path. At the classical level, Methods 0–3 compute slightly less than EAP-TLS Classic (0.182–0.202 vs. 0.241 ms), send 8–13× fewer bytes, and complete 3.0–3.3× sooner in the median. Cross-checks against pyca/cryptography confirmed all standard sizes.

6.3. Full 5G Core Path

To confirm operation over a complete 5G core, we ran the proposal through UERANSIM and Open5GS with microsecond timestamps in the UE, the SMF, the UPF relay, and the DN-AAA. The SMF runs secondary authentication as step 6 of PDU session establishment [45], before policy association, UPF selection, and N4 setup: the UE answers the EAP-Request/Identity with the anonymous NAI `@edhoc.internet`, and neither its SUPI nor a DN-specific identity appears in the NAS or RADIUS messages, EAP-Success travels in the PDU Session Establishment Accept, and an EAP-Failure in a Reject with cause #29 [70], as a packet capture confirms. Three series of 100 PDU sessions with secondary authentication alternated with three without it, each after five discarded warm-up sessions. All 300 sessions were accepted with EAP-Success, with matching MSKs, rotated keys, and the sizes and 15 round-trips of the RADIUS path. Table 6 gives medians with bootstrap CIs (4000 resamples). The added time is Request to Accept with minus without secondary authentication; the time at the UE runs from the first PDU Session Authentication Command to the Accept, so its 5G core row includes the policy, UPF, and N4 steps after the Access-Accept (about 5 ms), which also occur without it. Secondary authentication adds

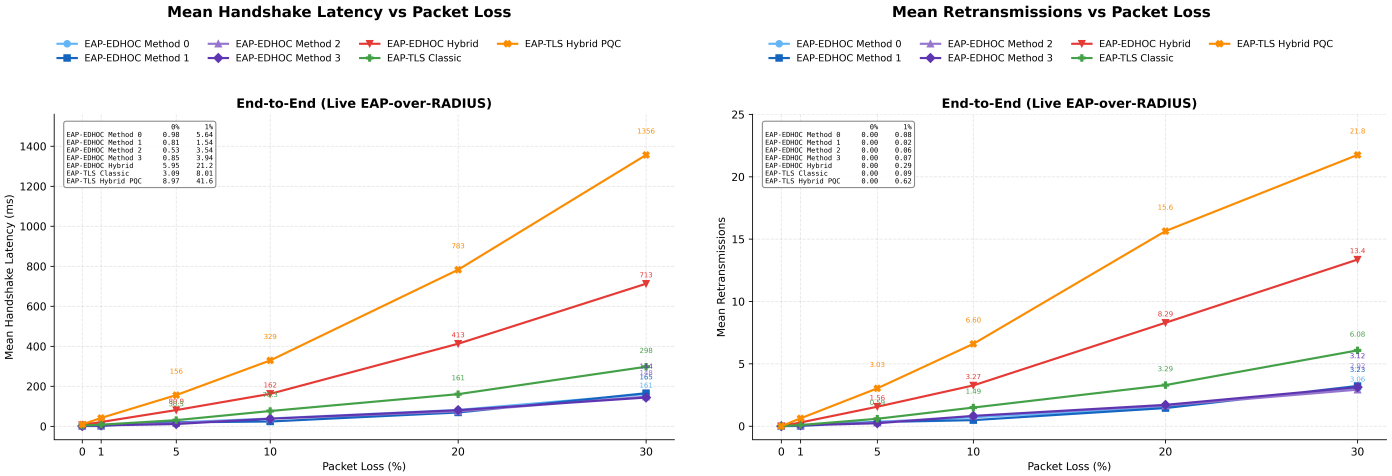
76 ms to PDU session establishment (21 vs. 97 ms), dominated by 5G core transport at about 4.6 ms per NAS round-trip, so each round-trip the proposal saves weighs more here than on the RADIUS segment. The DN-AAA takes 5.0 ms from the UPF relay, against 4.8 ms seen by the benchmark client with the core running idle, and the relay adds 2.7 ms for the SMF–UPF hop and 0.8 ms for RADIUS processing. The UE's write of NEXT takes longer here than on the RADIUS path because it is the first `fdatasync` after the disk has been idle between sessions, a cost of our VM's disk that a probe without the protocol reproduces. Dropping `Message3` or `Message4` once at the relay (10 runs each) reproduced the recovery of Section 4.4: the session with the drop was rejected without an EAP-Failure, so the UE kept its keys; after a lost `Message3` the next session was rejected with an EAP-Failure and the following one succeeded on the old keys (median 1.46 s, including the 1 s `reject_delay` of FreeRADIUS); after a lost `Message4` the next session succeeded on NEXT (101 ms). No run desynchronised or required re-provisioning.

Table 6. Proposal over the full 5G core path (300 sessions, ms).

Segment	Median	95% CI	Mean	p95
Added to PDU session establishment	75.9	[74.4, 77.1]	78.2	–
Secondary authentication at the UE	74.7	[73.4, 75.6]	79.5	112.0
5G core transport (UE, gNB, AMF, SMF)	69.4	[68.5, 70.5]	74.4	106.6
SMF ↔ UPF relay hop	2.7	[2.7, 2.8]	2.8	3.7
RADIUS processing in the UPF relay	0.8	[0.8, 0.8]	0.9	1.4
DN-AAA (FreeRADIUS, from the UPF relay)	5.0	[4.9, 5.1]	5.1	6.8
UE write of NEXT (before `Message3`)	2.84	[2.82, 2.87]	2.43	3.63
DN-AAA commit (before `Message4`)	0.58	[0.58, 0.59]	0.62	0.76
UE promotion (after `Message4`)	0.53	[0.52, 0.55]	0.59	0.79

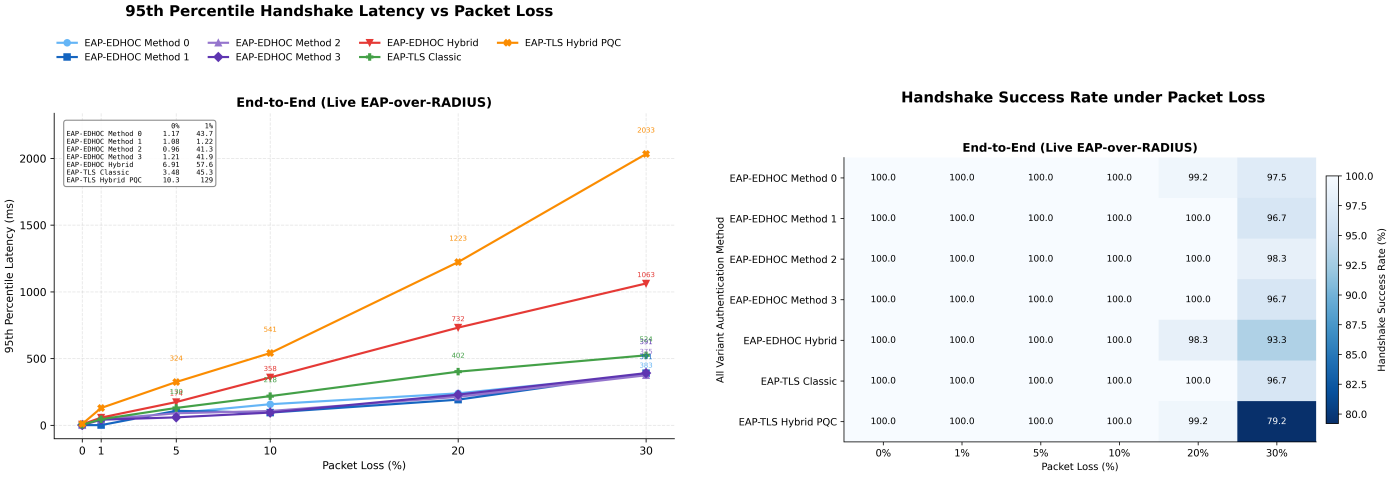
6.4. Behaviour under Packet Loss

Figures 9 and 10 show behaviour under emulated application-level loss with EAP-style retransmission (RTO 40 ms, up to 6 retransmissions, 120 trials per cell), measured before the $KeyID_B$ field was added; the round-trip count, which drives the loss behaviour, is unchanged at 15. Loss behaviour follows the round-trip count: if a round-trip succeeds with probability $p = (1 - L)^2$ and an exchange fails after 7 attempts, a handshake succeeds with probability $(1 - (1 - p)^7)^{n_{RT}}$, which at 30% loss predicts 87.4% for the proposal and 77.7% for EAP-TLS Hybrid PQC against the measured 93.3% and 79.2% (Wilson 95% intervals $[87.4, 96.6]$% and $[71.1, 85.5]$%; Fisher's exact test $p = 0.002$, while up to 20% loss the rates do not differ, $p = 1.0$). Methods 0–3 keep 100% up to 10% loss and 96.7–98.3% at 30%, as does EAP-TLS Classic (96.7%). Within each security level the EAP-EDHOC variant degrades less: at 30% loss Methods 0–3 average 144–165 ms against 298 ms for EAP-TLS Classic, and the proposal 713 ms against 1,357 ms, and the proposal has the lower mean latency of the hybrid pair at every loss level (21.2 vs. 41.6 ms at 1%). Fragmentation, not computation, is the main scalability concern.



(**a**) Mean latency. (**b**) Mean retransmissions.

Figure 9. Mean behaviour under packet loss: (**a**) latency; (**b**) retransmissions.

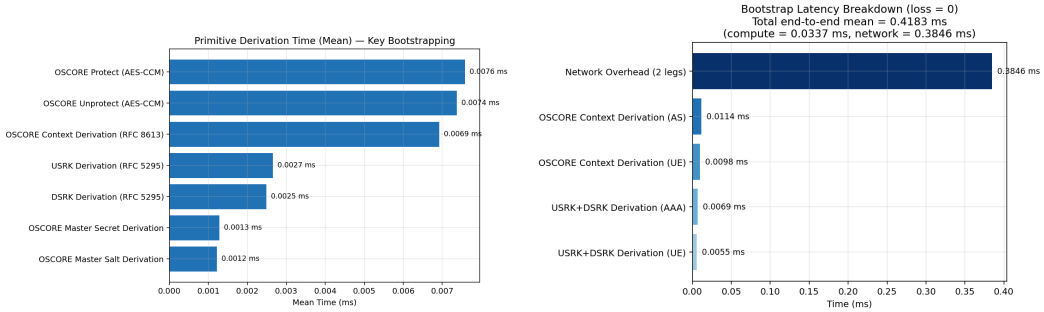


(**a**) 95th-percentile latency. (**b**) Success rate.

Figure 10. Packet loss: (**a**) 95th-percentile latency; (**b**) success rate.

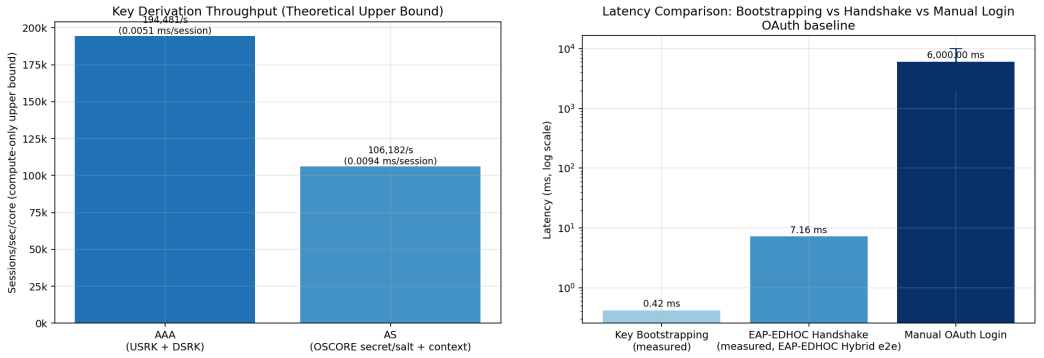
6.5. EMSK Key-Bootstrapping Cost

The bootstrapping chain is cheap (Figure 11): its primitives cost 0.0012–0.0076 ms, and of the 0.418 ms loss-free end-to-end bootstrap only 0.034 ms is key derivation, the rest being the AAA→AS and UE→AS network legs. A single core sustains 194,481 sessions/s at the AAA and 106,182 at the AS (Figure 12a), and at 6.5% of the handshake, bootstrapping is at least 4,700× faster than a 2–10 s manual OAuth 2.0 login (Figure 12b). The label `"dsrk@ietf.org"` is reserved by RFC 5295, whereas `"oscore-bootstrap@<specorg>"` is a proposed private-use label; distinct labels on the same Expand/Extract function give cryptographic separation, and *id_context* separates OSCORE sessions to one AS. RFC 5295 defines no DSRK transport; we meet the integrity, replay-protection, and key-wrapping requirements of RFC 5749 [71] with mutual TLS 1.3 between the DN-AAA and the AS.



(**a**) (**b**)

Figure 11. EMSK bootstrapping cost: (**a**) primitives; (**b**) end-to-end breakdown.



(**a**) (**b**)

Figure 12. EMSK bootstrapping: (**a**) throughput; (**b**) comparison with handshake and login.

7. Discussion

Symmetric guarantees, asymmetric costs. The protocol is role-symmetric in security, with independent signatures, KCI resistance, and key rotation on both sides, and balanced in computation (0.492 ms per party) and traffic (6,146 vs. 6,113 B), but its cost is asymmetric: computation is about 15% of the 6.383 ms handshake, and most of the rest is fragmentation over 15 round-trips. The extra computation against EAP-TLS Hybrid PQC is the price of symmetric rotation: both parties receive an encapsulation and must rotate, so key generation costs twice as much (0.167 vs. 0.084 ms; Table 5). Faster primitives would thus barely change latency, whereas a larger EAP fragment size and RADIUS attribute budget where the operator controls both ends, a larger MTU on the SMF–UPF–DN-AAA segment, or more compact PQC signatures would cut transport without touching the verified logic.

Design choices. A signature-free KEM method [26,27] would save the ML-DSA-44 sign/verify cost (0.223/0.060 ms) but lose the detached signature that gives KCI resistance independent of the KEM key, and with it the explicit-authentication symmetry. Unlike Hybrid-EDHOC [28], we keep `Message4`: it carries no key material but is the protected success indication that EAP-EDHOC mandates [12] and the point at which the UE switches keys, for one 16-byte fragment.

Threats to validity. The single-VM loopback figures are lower bounds, and the full path includes UERANSIM's simulated radio; loss is emulated at the application layer; and we do not rank Methods 0–3, whose medians differ by only 18 µs. The analysis is symbolic, with a discrete-log oracle for the quantum adversary; a diff-equivalence model of unlinkability across rotated runs did not terminate within our memory limit, so we do not claim that property; and the AAA→AS DSRK channel rests on a co-location/mTLS

assumption analogous to ERP [72]. Forward secrecy rests on erasure, which the recovery rules preserve on every modelled and tested path, and post-compromise security heals from the first run after a compromise. Re-authentication and handover are not evaluated.

8. Conclusions

We proposed a role-symmetric hybrid post-quantum EAP-EDHOC protocol for 5G secondary authentication, in which two opposite X-Wing encapsulations to one-time static keys, detached ML-DSA-44 signatures, and a two-sided key rotation with erasure give forward secrecy and post-compromise security without an ephemeral key pair. ProVerif shows that only the proposal combines quantum resilience, KCI resistance, forward secrecy, and post-compromise security, with identity concealment of ID_CRED_I unless the Responder's current one-time key is compromised before the run, and a lost message never forces re-provisioning. On Open5GS, UERANSIM, and FreeRADIUS the handshake exchanges 12,259 B, split evenly between the directions, and completes in 6.383 ms against 8.273 ms for EAP-TLS Hybrid PQC: it computes slightly more (0.985 vs. 0.894 ms) but needs about half the round-trips (15 vs. 28), and inside PDU session establishment on the full 5G core it adds 76 ms, mostly NAS transport. The exported EMSK bootstraps OSCORE keys for login-free application-server access. Future work covers compact PQC parameter sets, fragment-size and MTU tuning, mobility and handover, a computational proof, and IANA registration of the OSCORE-bootstrap label.

Author Contributions: Conceptualization, A.P.S.I., S.S. and I.Y.; methodology, A.P.S.I.; software, A.P.S.I., I.W.A.J.P. and H.K.; formal analysis, A.P.S.I. and Y.K.; investigation, A.P.S.I., Y.K., I.W.A.J.P. and H.K.; data curation, A.P.S.I., Y.K., I.W.A.J.P. and H.K.; writing original draft preparation, A.P.S.I. and S.S.; writing review and editing, A.P.S.I. and S.S.; supervision, I.Y. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by the Institute of Information & Communications Technology Planning & Evaluation(IITP) grant funded by the Korean government(MSIT) (No. RS-2024-00398312, Development of Quantum Security Based Device Identification and Test Verification Technologies for 5G Non-Public Network)

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The source code of the modified Open5GS, UERANSIM, and FreeRADIUS components, the EAP-EDHOC implementation, the ProVerif models, the benchmark scripts, and the raw measurement results supporting this study are openly available in the EdhocSecondaryAuth repository at https://github.com/adipanca/EdhocSecondaryAuth [34].

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

3GPP	3rd Generation Partnership Project
5G-AKA	5G Authentication and Key Agreement
AAA	Authentication, Authorization, and Accounting
AEAD	Authenticated Encryption with Associated Data
AMF	Access and Mobility Management Function
AS	Application Server
AUSF	Authentication Server Function
CBOR	Concise Binary Object Representation
COSE	CBOR Object Signing and Encryption
DN	Data Network
DSRK	Domain-Specific Root Key
EAP	Extensible Authentication Protocol
EDHOC	Ephemeral Diffie–Hellman Over COSE
EMSK	Extended Master Session Key
HKDF	HMAC-based Key Derivation Function
KCI	Key-Compromise Impersonation
KEM	Key Encapsulation Mechanism
ML-DSA	Module-Lattice-Based Digital Signature Algorithm
ML-KEM	Module-Lattice-Based Key-Encapsulation Mechanism
MSK	Master Session Key
MTU	Maximum Transmission Unit
OSCORE	Object Security for Constrained RESTful Environments
PQC	Post-Quantum Cryptography
RADIUS	Remote Authentication Dial-In User Service
SMF	Session Management Function
UE	User Equipment
UPF	User Plane Function
USRK	Usage-Specific Root Key

References

1. 3GPP TS 33.501. Security architecture and procedures for 5G System. https://www.3gpp.org/DynaReport/33501.htm, 2024. Release 18. Accessed on 5 March 2026.
2. 3GPP TS 23.501. System architecture for the 5G System (5GS). https://www.3gpp.org/DynaReport/23501.htm, 2024. Release 18. Accessed on 5 March 2026.
3. Edris, E.K.K.; Aiash, M.; Loo, J.K.K. Formal Verification and Analysis of Primary Authentication based on 5G-AKA Protocol. In Proceedings of the 3rd International Symposium on 5G Emerging Technologies (5GET), in conjunction with the 4th International Conference on Fog and Mobile Edge Computing (FMEC). IEEE, 2020, pp. 256–261.
4. Borgaonkar, R.; Hirschi, L.; Park, S.; Shaik, A. New Privacy Threat on 3G, 4G, and Upcoming 5G AKA Protocols. *Proceedings on Privacy Enhancing Technologies* **2019**, *2019*, 108–127.
5. IETF RFC 3748. Extensible Authentication Protocol (EAP). https://www.rfc-editor.org/rfc/rfc3748.html, 2004. Accessed on 5 March 2026.
6. Gong, S.; EL Azzaoui, A.; Cha, J.; Park, J.H. Secure Secondary Authentication Framework for Efficient Mutual Authentication on a 5G Data Network. *Applied Sciences* **2020**, *10*, 727.
7. IETF RFC 5216. The EAP-TLS Authentication Protocol. https://www.rfc-editor.org/rfc/rfc5216.html, 2008. Accessed on 5 March 2026.
8. IETF RFC 9190. EAP-TLS 1.3: Using the Extensible Authentication Protocol with TLS 1.3. https://www.rfc-editor.org/rfc/rfc9190.html, 2022. Accessed on 29 October 2026.
9. IETF RFC 9846. The Transport Layer Security (TLS) Protocol Version 1.3. https://www.rfc-editor.org/rfc/rfc9846.html, 2026. Obsoletes RFC 8446. Accessed on 29 October 2026.
10. IETF RFC 9528. Ephemeral Diffie-Hellman Over COSE (EDHOC). https://www.rfc-editor.org/rfc/rfc9528.html, 2024. Accessed on 1 February 2026.
11. IETF RFC 9529. Traces of Ephemeral Diffie-Hellman Over COSE (EDHOC). https://www.rfc-editor.org/rfc/rfc9529.html, 2024. Accessed on 2 February 2026.

12. IETF Internet-Draft. Extensible Authentication Protocol (EAP) using EDHOC (EAP-EDHOC). https://datatracker.ietf.org/doc/draft-ietf-emu-eap-edhoc/, 2025. Accessed on 5 March 2026.
13. Fedrecheski, G.; Vučinić, M.; Watteyne, T. Performance Comparison of EDHOC and DTLS 1.3 in Internet-of-Things Environments. In Proceedings of the Wireless Communications and Networking Conference (WCNC) 2024. IEEE, 2024.
14. Pérez, E.L.; Selander, G.; Mattsson, J.P.; Watteyne, T.; Vučinić, M. EDHOC is a New Security Handshake Standard: An Overview of Security Analysis. *Computer* **2024**, *57*, 101–110.
15. Shor, P. Algorithms for quantum computation: discrete logarithms and factoring. In Proceedings of the Proceedings 35th Annual Symposium on Foundations of Computer Science. IEEE, 1994, pp. 124–134.
16. NIST. Post-Quantum Cryptography. https://csrc.nist.gov/projects/post-quantum-cryptography. Accessed on 26 March 2026.
17. Migration to Post-Quantum Cryptography. NIST Crypto Agility Guide. Accessed on 14 February 2026.
18. ANSSI views on the Post-Quantum Cryptography transition. https://cyber.gouv.fr/en/publications/anssi-views-post-quantum-cryptography-transition. Accessed on 14 February 2026.
19. IETF Internet-Draft. Terminology for Post-Quantum Traditional Hybrid Schemes. https://www.ietf.org/archive/id/draft-ietf-pquip-pqt-hybrid-terminology-04.html, 2024. Accessed on 10 February 2026.
20. Bindel, N.; Brendel, J.; Fischlin, M.; Goncalves, B.; Stebila, D. Hybrid Key Encapsulation Mechanisms and Authenticated Key Exchange. In Proceedings of the PQCrypto 2019. Springer, 2019, pp. 206–226.
21. Barbosa, M.; Connolly, D.; Duarte, J.D.; Kaiser, A.; Schwabe, P.; Varner, K.; Westerbaan, B. X-Wing: The Hybrid KEM You've Been Looking For. *IACR Communications in Cryptology* **2024**, *1*. https://doi.org/10.62056/a3qj89n4e.
22. IETF Internet-Draft. X-Wing: General-Purpose Hybrid Post-Quantum KEM (draft-connolly-cfrg-xwing-kem-11). https://www.ietf.org/archive/id/draft-connolly-cfrg-xwing-kem-11.html, 2026. Accessed on 4 November 2026.
23. IETF RFC 2865. Remote Authentication Dial In User Service (RADIUS). https://www.rfc-editor.org/rfc/rfc2865.html, 2000. Accessed on 5 March 2026.
24. IETF RFC 3579. RADIUS (Remote Authentication Dial In User Service) Support For Extensible Authentication Protocol (EAP). https://www.rfc-editor.org/rfc/rfc3579.html, 2003. Accessed on 5 March 2026.
25. Fraile, L.P.; Tasopoulos, G.; Koulamas, C.; Zhao, R.K.; Sultan, N.H.; Regazzoni, F.; Fournaris, A.P. Enabling Quantum-Resistant EDHOC: Design and Performance Evaluation. *IEEE Access* **2025**, *13*, 75861–75884.
26. Fraile, L.P.; Koulamas, C.; Fournaris, A.P. Reinventing EDHOC for the Post-Quantum Era. *IEEE Access* **2025**, *13*, 196622–196639. https://doi.org/10.1109/ACCESS.2025.3633843.
27. IETF Internet-Draft. KEM-based Authentication for EDHOC. https://www.ietf.org/archive/id/draft-pocero-authkem-edhoc-02.html, 2026. Accessed on 26 March 2026.
28. Shin, S.; Iskandar, A.P.S.; Ko, Y.; You, I. A Hybrid EDHOC Protocol. In Proceedings of the The 9th International Conference on Mobile Internet Security (MobiSec 2025), 2025.
29. Blanchet, B. An Efficient Cryptographic Protocol Verifier Based on Prolog Rules. In Proceedings of the CSFW, 2001.
30. Blanchet, B. Automated Verification of Correspondences for Security Protocols. *Journal of Computer Security* **2016**.
31. Lee, S. Open5GS: Open Source implementation of 5G Core and EPC. https://open5gs.org/, 2024. Accessed on 5 March 2026.
32. Güngör, A. UERANSIM: Open source 5G UE and RAN (gNodeB) simulator. https://github.com/aligungr/UERANSIM, 2024. Accessed on 5 March 2026.
33. The FreeRADIUS Server Project. FreeRADIUS: The world's most popular RADIUS Server. https://www.freeradius.org/, 2024. Accessed on 5 March 2026.
34. Iskandar, A.P.S. EdhocSecondaryAuth: Hybrid Post-Quantum EAP-EDHOC Secondary Authentication over Open5GS, UERANSIM, and FreeRADIUS (Source Code, ProVerif Models, and Benchmark Data). https://github.com/adipanca/EdhocSecondaryAuth, 2026. Accessed on 13 October 2026.
35. Jacomme, C.; Klein, E.; Kremer, S.; Racouchot, M. A comprehensive, formal and automated analysis of the EDHOC protocol. In Proceedings of the USENIX Security Symposium 2023. USENIX Association, 2023, pp. 5881–5898.
36. Norrman, K.; Sundararajan, V.; Bruni, A. Formal Analysis of EDHOC Key Establishment for Constrained IoT Devices. In Proceedings of the SECRYPT 2021. SCITEPRESS, 2021, pp. 210–221.
37. Kim, J.; Duguma, D.G.; Lee, S.; Kim, B.; Lim, J.; You, I. Scrutinizing the Vulnerability of Ephemeral Diffie–Hellman over COSE (EDHOC) for IoT Environment Using Formal Approaches. *Mobile Information Systems* **2021**, *2021*.
38. Günther, F.; Mukendi, M.I.T. Careful with MAc-then-SIGn: A Computational Analysis of the EDHOC Lightweight Authenticated Key Exchange Protocol. In Proceedings of the EuroS&P 2023. IEEE, 2023, pp. 773–796.
39. Cremers, C.; Dax, A.; Medinger, N. Keeping Up with the KEMs: Stronger Security Notions for KEMs and Automated Analysis of KEM-based Protocols. In Proceedings of the CCS 2024. ACM, 2024, pp. 1046–1060.
40. Cottier, B.; Pointcheval, D. Security Analysis of Improved EDHOC Protocol. In Proceedings of the Foundations and Practice of Security (FPS) 2022. Springer, 2022, pp. 3–18.

41. Ferreira, L. Computational Security Analysis of the Full EDHOC Protocol. In Proceedings of the CT-RSA 2024. Springer, 2024, pp. 25–48.
42. Shin, S.; et al. A Hybrid EDHOC Protocol Post-Quantum Secure against Key Compromise Impersonation. In Proceedings of the Proceedings of eBISION 2026, 2026.
43. IETF RFC 10024. Post-Quantum Traditional (PQ/T) Hybrid Key Agreement Mechanisms for TLS 1.3. https://www.rfc-editor.org/rfc/rfc10024.html, 2026. Accessed on 29 October 2026.
44. IETF Internet-Draft. Use of ML-DSA in TLS 1.3 (draft-ietf-tls-mldsa-05). https://www.ietf.org/archive/id/draft-ietf-tls-mldsa-05.html, 2026. Accessed on 29 October 2026.
45. 3GPP TS 23.502. Procedures for the 5G System (5GS). https://www.3gpp.org/DynaReport/23502.htm, 2024. Release 18. Accessed on 30 October 2026.
46. IETF RFC 5295. Specification for the Derivation of Root Keys from an Extended Master Session Key (EMSK). https://www.rfc-editor.org/rfc/rfc5295.html, 2008. Accessed on 5 March 2026.
47. IETF RFC 8613. Object Security for Constrained RESTful Environments (OSCORE). https://www.rfc-editor.org/rfc/rfc8613.html, 2019. Accessed on 5 March 2026.
48. NIST FIPS 203. Module-Lattice-Based Key-Encapsulation Mechanism Standard. https://csrc.nist.gov/pubs/fips/203/final, 2024. Accessed on 2 February 2026.
49. CRYSTALS. https://pq-crystals.org/. Accessed on 10 February 2026.
50. ETSI TS 103 744. CYBER; Quantum-safe Hybrid Key Exchanges. https://www.etsi.org/deliver/etsi_ts/103700_103799/103744/01.01.01_60/ts_103744v010101p.pdf, 2020. Accessed on 12 February 2026.
51. Angel, Y.; Dowling, B.; Hülsing, A.; Schwabe, P.; Weber, F. Post Quantum Noise. In Proceedings of the CCS 2022. ACM, 2022, pp. 97–109.
52. IETF RFC 9954. Hybrid Key Exchange in TLS 1.3. https://www.rfc-editor.org/rfc/rfc9954.html, 2026. Accessed on 29 October 2026.
53. IETF RFC 9370. Multiple Key Exchanges in the Internet Key Exchange Protocol Version 2 (IKEv2). https://www.rfc-editor.org/rfc/rfc9370.html, 2023. Accessed on 11 February 2026.
54. IETF Internet-Draft. Composite ML-KEM for Use in the Internet X.509 Public Key Infrastructure and CMS. https://www.ietf.org/archive/id/draft-ietf-lamps-pq-composite-kem-04.html, 2024. Accessed on 22 February 2026.
55. The PQXDH Key Agreement Protocol. https://signal.org/docs/specifications/pqxdh/. Accessed on 10 February 2026.
56. Fiedler, R.; Günther, F. Security Analysis of Signal's PQXDH Handshake. https://eprint.iacr.org/2024/702, 2024. Cryptology ePrint Archive: Paper 2024/702.
57. Dolev, D.; Yao, A.C. On the Security of Public Key Protocols. *IEEE Transactions on Information Theory* **1983**, *29*, 198–208. https://doi.org/10.1109/TIT.1983.1056650.
58. IETF RFC 7542. The Network Access Identifier. https://www.rfc-editor.org/rfc/rfc7542.html, 2015. Accessed on 4 November 2026.
59. Krawczyk, H. SIGMA: The 'SIGn-and-MAc' Approach to Authenticated Diffie-Hellman and Its Use in the IKE Protocols. In Proceedings of the CRYPTO 2003. Springer, 2003, pp. 400–425.
60. NIST FIPS 204. Module-Lattice-Based Digital Signature Standard. https://csrc.nist.gov/pubs/fips/204/final, 2024. Accessed on 13 February 2026.
61. Dolev, D.; Yao, A. On the Security of Public Key Protocols. In Proceedings of the IEEE Symposium on Foundations of Computer Science, 1983.
62. Cremers, C.; Horvat, M.; Hoyland, J. A Symbolic Analysis of TLS 1.3. In Proceedings of the IEEE Symposium on Security and Privacy, 2017.
63. ISO/IEC 29128: Information technology – Security techniques – Verification of cryptographic protocols. International Organization for Standardization (ISO), ISO/IEC 29128, 2011.
64. The Libsodium Authors. Libsodium: A modern and easy-to-use crypto library. https://libsodium.org, 2024. Accessed: 2026-04-03.
65. Mbed TLS Contributors. Mbed TLS: Lightweight, modular, and portable cryptographic library. https://github.com/Mbed-TLS/mbedtls, 2024. Accessed: 2026-04-03.
66. PQClean Project Contributors. PQClean: Clean, portable, tested implementations of post-quantum cryptography. https://github.com/PQClean/PQClean, 2024. Accessed: 2026-04-03.
67. Open Quantum Safe Project. liboqs: C library for prototyping and experimenting with quantum-resistant cryptography. https://github.com/open-quantum-safe/liboqs, 2024. Accessed on 5 March 2026.
68. Open Quantum Safe Project. oqs-provider: Open Quantum Safe Provider for OpenSSL 3. https://github.com/open-quantum-safe/oqs-provider, 2024. Accessed on 17 March 2026.

69. IETF RFC 9881. Internet X.509 Public Key Infrastructure: Algorithm Identifiers for the Module-Lattice-Based Digital Signature Algorithm (ML-DSA). https://www.rfc-editor.org/rfc/rfc9881.html, 2025. Accessed on 29 October 2026.
70. 3GPP TS 24.501. Non-Access-Stratum (NAS) protocol for 5G System (5GS); Stage 3. https://www.3gpp.org/DynaReport/24501.htm, 2024. Release 18. Accessed on 30 October 2026.
71. IETF RFC 5749. Distribution of EAP-Based Keys for Handover and Re-Authentication. https://www.rfc-editor.org/rfc/rfc5749.html, 2010. Accessed on 5 March 2026.
72. IETF RFC 6696. EAP Extensions for the EAP Re-authentication Protocol (ERP). https://www.rfc-editor.org/rfc/rfc6696.html, 2012. Accessed on 15 August 2026.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.